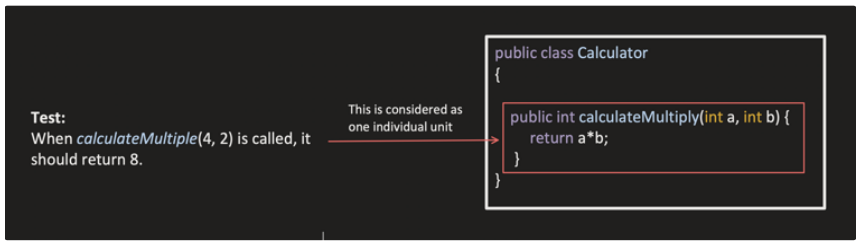
Unit Testing Introduction

What is Unit Testing

It's a testing method, where individual unit (like method or function) is tested in isolation.



- And goal is to test it in isolation, means, we need to test functionality of *calculateMultiply()* method only, not any other method functionality.

- In below, *calculateMultiply()* method calls *NumberUtils* class *multiply*() method, so we don't want to test *NumberUtils multiply*() method.

- So we mock this call *numUtil.multiply(a, b)*.
  Mocking means: instead of invoking the real method, we tell the test framework, "when this method is called, just return this predefined value without executing the actual logic."

Concept && Coding

```java
public class Calculator
{
    private NumberUtils numUtil;

    public Calculator(NumberUtils numUtil) {
        this.numUtil= numUtil;
    }

    public int calculateMultiply(int a, int b) {
         // calling another class function
         return numUtil.multiply(a, b);
     }
}



public class NumberUtils
{
       public int multiply(int a, int b) {
        return  a*b;
     }
}
```

<u>Advantages of Unit Testing</u>

It's a "FIRST LINE OF DEFENCE"

Early Bug detection:

If a method returns wrong output (e.g. 14 instead of 12), you catch it while coding — not after release.

Refactor Confidently:

When you change code, unit test case warn you if something breaks — so you don't break features silently.

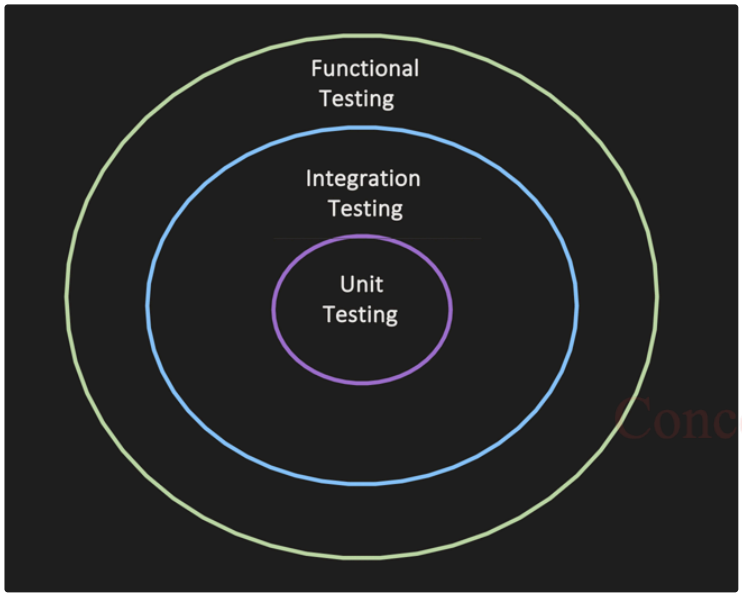
Documentation:

Unit test case show how a method behaves — no need to read all the code to understand what it does.

Save Cost:

Fixing bugs during development is way cheaper than fixing them after app goes live.

Lets understand the difference between: Unit vs Integration vs Functional Testing



Concept && Coding

Unit : we have already seen above, it's a testing of an individual unit (or method)

Integration Testing:

Testing of multiple modules or components together, to test if system behaves properly when wired together.

(pls read above line again: Goal is not to test end to end user behavior, goal is just if multiple modules or system wired together, does intended behavior is achieved or not).

Use case 1:

Wired multiple modules of 1 component.

Service → Repository → DB

- You invoke a method in the Service layer
- Verify that Spring Dependency Injection works correctly
- The flow should reach the database layer
- Your test can assert that data was persisted properly into the DB

Stub or Mock the external service calls, as goal is to test multiple modules of 1 component only, not multiple microservices.

Use case 2:

Integration with multiple components (not all the components present in the system).

- **Service A → Service B**
- Validate that Service A successfully calls Service B via REST (RestTemplate, Feign etc)
- We can test things like response mapping, HTTP error handling, etc.

- **Service A → publish to Kafka**
- Validate that a message is successfully published to the Kafka topic

- **Service A → send mail**
- Validate that **email payload is built and sent correctly**

Concept && Coding

Functional Testing:

End to End testing of a feature business functionality, verifying it works as expected.

Use case 1:

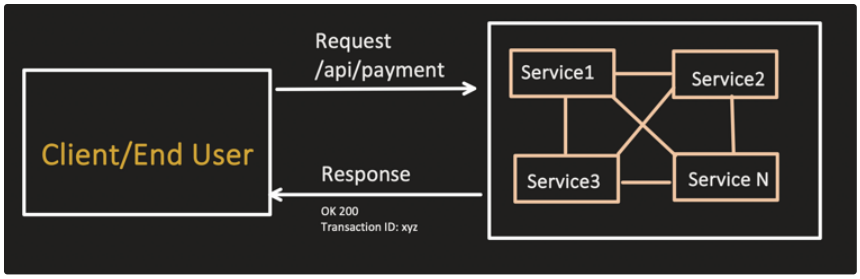
For 1 component, doing Functional testing. End to End testing from our client perceptive.

API → Controller → Service (invokes Real external service call (REST call) → DB

Use case 2:

End to End testing from end user perceptive, like how end user interact with system (from UI).

May include frontend, backend, DB, Kafka etc.
All components which are involved in a feature, need to deploy they components on a stage or QA, and then testing is done.



JUnit with Mockito is most popular. And will use it in upcoming videos

Concept && Coding